

AT-02

S. Ashwag Ahamed

Reg No: 192572070

Sub:- Data structure

Code: CSA0306

An operating system manages dynamic memory allocation. Analyze how linked list can represent free memory blocks. Compare this with array-based memory management in terms of flexibility, fragmentation and performance and justify the better approach.

Ans

An operating system uses memory to run program. A linked list is used to store free memory block because it is dynamic. Each node stores the memory block size and the address of released nodes can be added or removed easily without shifting the order of data.

An array based memory management stores memory blocks in fixed position. It is simple and provides fast access using indexes, but the size is fixed. Difficult because element must be shifted which wastes time.

Comparison

Linked list

- Dynamic size
- Easy insertion and deletion.
- Less memory wastage
- Better for dynamic memory.

Array

- Fixed size.
- Difficult insertion and deletion.
- Memory wastage is more.

• Better for Fixed data.

Conclusion

Linked list are the better choice for operating system memory management because they provide flexibility, reduce memory wastage, and efficiently handle dynamic memory allocation.

2. E - Learning platform

a. primitive and non-primitive data structure.

An e-learning platform stores information about students, courses and discussions.

- Primitive Data structure: int, float, char, & boolean.
- student ID - int
- student name → char[]
- Course fee → float.
- course status → boolean.
- Non primitive Data structure:
 - Array : stores course details
 - linked list: stores student records because students can join or leave.
 - Queue : Handles discussion messages in order.

b. Advantages of suitable structure

- Primitive data type are simple and require less memory.

- Array provides fast access to course details.
- Linked list allows easy insertion and deletion of student records.
- Queue process discussion messages in the current order.

Conclusion

The best choice is linked list for students, Array for course, & queue for discussion because they improve efficiency and make it data management easy.

3 Ride-sharing system

A ride sharing application receives ride request assign drivers & find the shortest route.

Data structure used

ADT: Queue is used to store ride request.

- Linear Data structure: Queue manages users waiting for rides.
- Non linear data structure: Graph represents roads and location.

Performance Analysis

- Queue insertion and deletion take $O(1)$ time.
- Graph search using BFS or DFS takes $O(V+E)$ time.

• Space Complexity depends on the number of users and roads.

Justification

Using a queue ensures that ride requests are handled in the order they arrive. A graph helps find the shortest route b/w the driver & passenger.

Conclusion

A combination of queue and Graph provides fast driver allocation and efficient route planning.

4. Linear search and Binary search

A system searches records stored in sorted order.

Linear search

- checks each element one by one.
- works on both sorted and unsorted data.
- Best case: $O(1)$
- Average case: $O(n)$
- Worst case: $O(n)$

Binary search

- works only on sorted data.
- compares the middle element and repeatedly divides the search area into two halves.
- Best case: $O(1)$
- Average case: $O(\log n)$
- Worst case: $O(\log n)$

Comparison

Linear search

- works on any list.
- slower.
- Time Complexity: $O(n)$
- more comparisons.

Binary search

- Requires sorted list.
- faster.
- Time Complexity: $O(\log n)$
- fewer comparisons.

Conclusion

Binary search is more efficient because it requires fewer comparisons and searched much faster in sorted data.

5. Time - space Trade off

A mobile application must process large amount of data with using limited memory

The time space trade off means improving execution time by using more memory or saving memory by taking more execution time

Compromise

- more memory, less time: caching stores frequently used data, making the application faster but using extra memory.

Advantages

- Faster algorithms improve user experience.
- memory efficient algorithms are useful for device with limited RAM.

Conclusion

For mobile application, a balanced approach is best solution. It should use memory efficiently while providing fast performance, giving a better user experience.
